Simulation setting and silhouette score analysis

Loading libraries

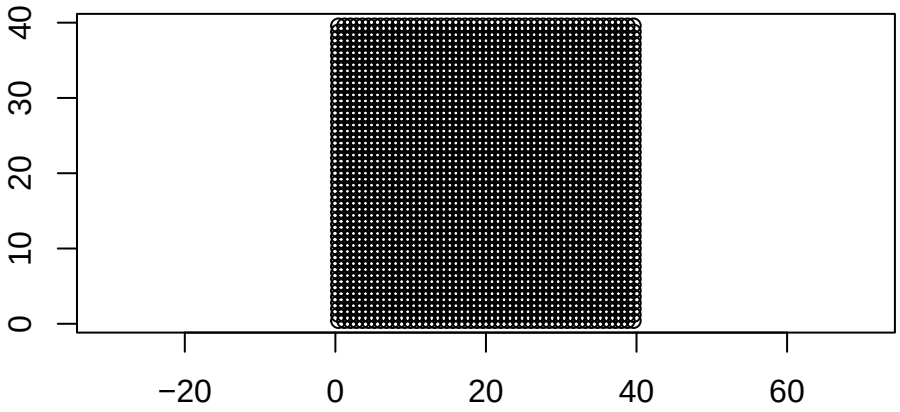
```
library(sf)
library(ggplot2)
library(spdep)
library(Matrix)
library(MASS)
library(dplyr)
library(sp)
library(adespatial)
library(cluster)
```

Simulate coordinates and regions

- Generating 'n' coordinates inside dim x dim grid

```
# Grid dimension
dim = 40

# Number of points to simulate
n = 2500
nrw = 50
ncl = 50

# Defining the domain size
sfc = st_sfc(
  st_polygon(list(rbind(c(0,0), c(dim,0), c(dim,dim), c(0,dim), c(0,0))))
  )

# Simulating the coordinates
```

```r
coordinates = st_make_grid(sfc, n=c(nrw,ncl), what = "centers")

plot(coordinates, axes = TRUE)
```



Defining different regions inside the polygon

```r
# Defining the circle 1
center1 <- c(33,8)
radius1 <- 5
circle1 <- st_buffer(st_sfc(st_point(center1)), dist = radius1)

# Defining the circle 2
center2 <- c(8,33)
radius2 <- 5
circle2 <- st_buffer(st_sfc(st_point(center2)), dist = radius2)

# Defining the quadrangle
corner_coordinates_1 <- list(rbind(c(3,3), c(12,3), c(12,12), c(3,12), c(3,3)))
square <- st_polygon(corner_coordinates_1)

# Defining the triangle
corner_coordinates_2 <- list(rbind(c(28,31), c(38,31), c(33,38), c(28,31)))
triangle <- st_polygon(corner_coordinates_2)

# Defining center larger circle and smaller circle at the center
centers <- c(20,20)
radius_l <- 8
radius_s <- 5
```

```r
large_circle <- st_buffer(st_sfc(st_point(centers)), dist = radius_l)
small_circle <- st_buffer(st_sfc(st_point(centers)), dist = radius_s)

# Create a ring by subtracting the smaller circle from the larger circle
ring <- st_difference(large_circle, small_circle)
```

Capturing the coordinates inside different regions

```r
# Circle 1
circle_1_coords <- st_intersection(coordinates, circle1)
# Quadrange
quad_coords <- st_intersection(coordinates, square)
# Circle 2
circle_2_coords <- st_intersection(coordinates, circle2)
# Triangle
tri_coords <- st_intersection(coordinates, triangle)
# Ring
ring_coords <- st_intersection(coordinates, ring)
# Circle inside the ring
circle_s_coords <-st_intersection(st_intersection(coordinates, small_circle))
```

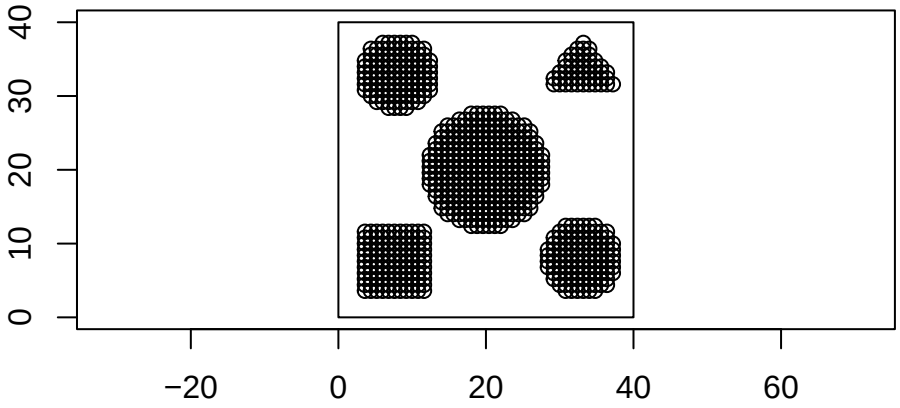
Capturing the number of coordinates inside each region

```r
num_circle_1 <- length(circle_1_coords)
num_quad <- length(quad_coords)
num_circle_2 <- length(circle_2_coords)
num_tri <- length(tri_coords)
num_ring <- length(ring_coords)
num_circle_s <- length(circle_s_coords)
```

Plotting all the captured regions

```r
cap_coords <- st_sf(
  coords = c(
    circle_1_coords,
    circle_2_coords,
```

```
    quad_coords,
    tri_coords,
    circle_s_coords,
    ring_coords
  )
)
plot(sfc, axes = TRUE)
plot(cap_coords, add = TRUE)
```



Coordinates in the un-captured region (noise)

```
# Function to create an "sf object" of the union of the shapes
union_sf <- function(sf_objects) {
  result <- sf_objects[[1]]
  for (i in 2:length(sf_objects)) {
    result <- st_union(result, sf_objects[[i]])
  }
  return(result)
}

sf_list <- list(circle1, circle2, square, triangle, large_circle)

union_result <- union_sf(sf_list)

# Coordinates of the noise
non_coords <- st_sf(
  coords = st_difference(coordinates, union_result)
)
```

```r
plot(sfc, axes = TRUE)
plot(non_coords, add = TRUE)
```

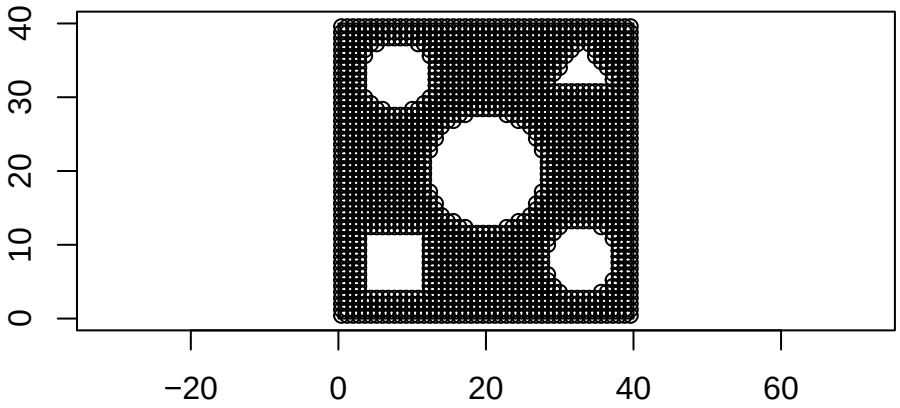
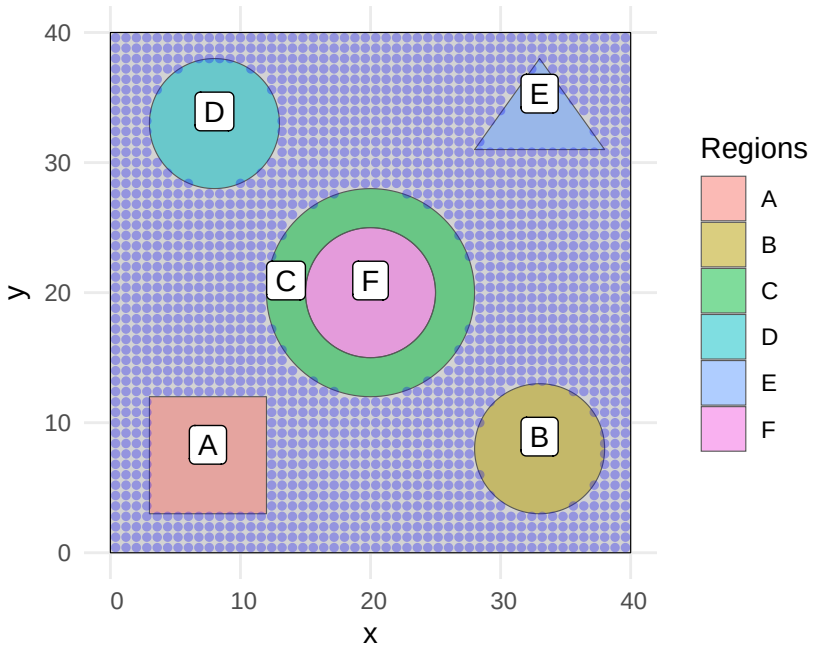


Image of the simulation setup

```r
# Ensure all shapes are in sfc_POLYGON format
circle1_sfc <- st_sfc(circle1)
circle2_sfc <- st_sfc(circle2)
square_sfc <- st_sfc(square)
triangle_sfc <- st_sfc(triangle)
center_sfc <- st_sfc(small_circle)
ring_sfc <- st_sfc(ring)



# Create an sf object with all shapes and labels
shapes <- st_sf(
  geometry = st_sfc(circle1_sfc[[1]], circle2_sfc[[1]], square_sfc[[1]], triangle_sfc[[1]], c
  label = c("B", "D", "A", "E", "F","C")
)

uncaptured_coords <- st_as_sf(non_coords)

# Plot the shapes with labels using ggplot2
ggplot() +
  geom_sf(data = sfc, fill = "lightgray", color = "black") +  # Plot the domain
  geom_sf(data = shapes, aes(fill = label), alpha = 0.5) +  # Plot captured shapes
```

```
  geom_sf(data = uncaptured_coords, aes(geometry = coords), color = "blue", alpha = 0.3, size
  geom_sf_label(data = shapes, aes(label = label), color = "black", nudge_y = 0.8, nudge_x =
  theme_minimal() +
  labs(fill = "Regions") +
  theme(legend.position = "right")
```



Simulate data

Function to simulate from CAR model

```
simulate_CAR <- function(
    n,
    coords,
    k,
    mu,
    tau_sq,
    eta
    ){# univariate CAR model

  # Creating k-nearest neighbors
  nb_simulation <- knn2nb(
```

```
    knearneigh(
      coords,
      k
      )
    )

  nb_sym <- make.sym.nb(nb_simulation)

  # Weight matrix for a neighbor list - W is row standardised
  w_mat <- nb2mat(
    nb_sym,
    style = "W"
    )


  # Defining C matrix
  c <- eta*w_mat

  # Defining mean matrix
  mu_mat <- rep(mu,n)

  # Defining sigma matrix
  identity <- diag(seq(1,1,length=n))
  sigma_mat <-  tau_sq*solve(identity-c)

  # Simulating from joint distribution of the variables from CAR model (MVN)
  z_CAR <- mvrnorm(n=1, mu_mat, sigma_mat)
  z <- as.vector(z_CAR)
  return(z)
}
```

Function to simulate noise

```
simulate_noise <- function(n, coords, p, mu, sigma){

  # Initiating mean and sigma
  mu_mat <- rep(mu,p)

  sigma_mat <- diag(rep(sigma, p))
```

```
  # Simulating data from the multivariate normal distribution
  noise <- mvrnorm(n=n,mu=mu_mat, Sigma = sigma_mat)
  colnames(noise) <- paste0("Z",1:p)
  data <- cbind(coords, noise)

  return(data)
}
```

Parameters set 1

- Simulating data for circle1, quadrangle and ring from the same distribution (CAR model)

```
# Number of neighbors for simulation
k_sim <- 8

# Number of variables(for each coordinate)
p <- 5

# Model parameters for Square, Circle 1 and Ring
mu_1 <- 4
tau_sq_1 <- 1
eta_1 <- 0.8

# Set of regions
regions_1 <- list(
    st_sf(coords = circle_1_coords),
    st_sf(coords = quad_coords),
    st_sf(coords = ring_coords)
)

# Number of coordinates for each region
regions_1_num <- c(
  num_circle_1,
  num_quad,
  num_ring
)

# Initialize an empty list to store the results
results_list_1 <- list()

for (i in 1:length(regions_1_num)) {
```

```
  sim_data_1 <- data.frame(
    lapply(1:p, function(x) simulate_CAR(regions_1_num[i], regions_1[[i]], k_sim, mu_1, tau_s
    coords = regions_1[[i]]
  )


  names(sim_data_1)[1:p] <- paste0("Z", 1:p)

  results_list_1[[i]] <- sim_data_1
}

data_1 <- bind_rows(results_list_1)
```

Parameter set 2

- Simulating data for triangle, circle 2 and inner ring circle

```
# Model parameters for triangle, Circle 2 and inner ring circle
mu_2 <- 10
tau_sq_2 <- 1
eta_2 <- .8

# Set of regions
regions_2 <- list(
    st_sf(coords = circle_2_coords),
    st_sf(coords = tri_coords),
    st_sf(coords = circle_s_coords)
)

# Number of coordinates for each region
regions_2_num <- c(
  num_circle_2,
  num_tri,
  num_circle_s
)

# Initialize an empty list to store the results
results_list_2 <- list()

for (i in 1:length(regions_2_num)) {
```

```
  sim_data_2 <- data.frame(
    lapply(1:p, function(x) simulate_CAR(regions_2_num[i], regions_2[[i]], k_sim, mu_2, tau_s
    coords = regions_2[[i]]
  )

  names(sim_data_2)[1:p] <- paste0("Z", 1:p)

  results_list_2[[i]] <- sim_data_2
}

data_2 <- bind_rows(results_list_2)
```

Simulating noise

```
mu_noise <- 0
sigma_noise <- 1
noise_coords <- non_coords
num_noise <- n - sum(regions_1_num,regions_2_num)
data_noise <- simulate_noise(num_noise, noise_coords, p, mu_noise, sigma_noise)
data_noise <- cbind(Regions = "Noise", data_noise)
```

Data

- Creating a dataframe with simulated and the respective shape

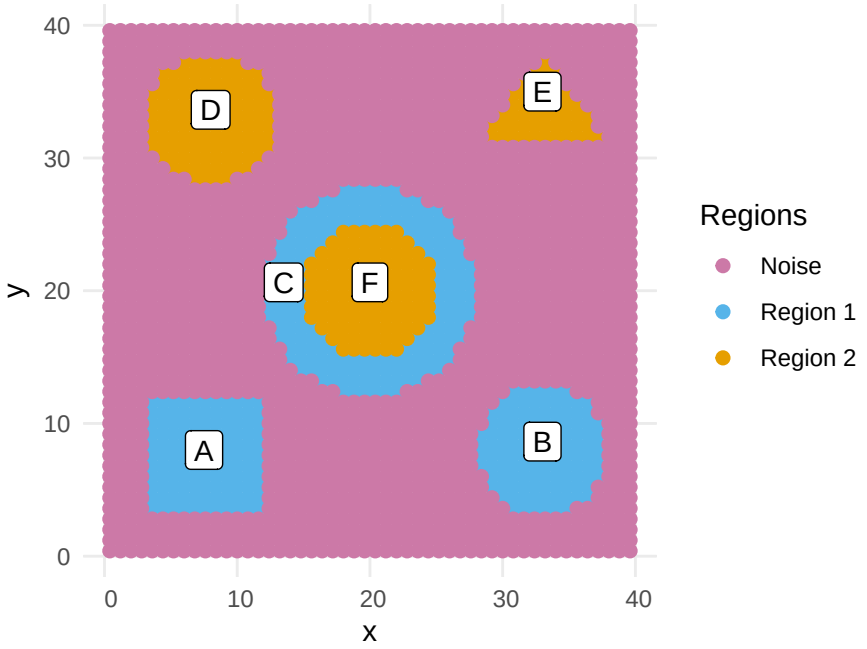
```
# Combining the two lists
results_list <- c(results_list_1, results_list_2)

# Name the result_list in the order of the simulated shapes
names(results_list) <- c(rep("Region 1",3), rep("Region 2", 3))

# convert the list to a data frame with a column for the name of the region
data_regions <- results_list |>
  bind_rows(.id = "Regions")

data_regions<- st_as_sf(data_regions, sf_column_name = "coords")

# Merge noise and the clusters
data_regions <- rbind(data_regions,data_noise)
```

Plotting the setup

```
cb_palette <- c("#CC79A7","#56B4E9", "#E69F00", "#009E73", "#0072B2", "#D55E00","#F0E442")

g1 <- ggplot() +  # Plot captured shapes
  geom_sf(data = data_regions, aes(geometry = coords,, color = Regions), size = 2) +  # Trans
  geom_sf_label(data = shapes, aes(label = label), color = "black", nudge_y = 0.5) +  # Add l
  scale_color_manual(values = cb_palette) +
  theme_minimal() +
  labs(fill = "Regions") +
  theme(legend.position = "right")
```

```
g1
```

```
Ignoring unknown labels:
* fill : "Regions"
```




```
# ggsave("Simulation_Setup.png", plot = g1, width = 7, height = 5, units = "in")
```

Modified silhouette score

```r
calculate_ai <- function(i, clusters, dist_matrix) {
  # Get the current cluster for observation i
  current_cluster <- clusters[i]

  # Get the observation numbers in the same cluster as i
  current_cluster_obs <- which(clusters == current_cluster)

  # Exclude observation i from the list of current cluster observations
  current_cluster_obs <- current_cluster_obs[current_cluster_obs != i]

  if (length(current_cluster_obs) == 0) {
    return(0)
  }

  # Calculate the average distance between i and other observations in the same cluster
  a_i <- mean(dist_matrix[i, current_cluster_obs])

  return(a_i)
}

calculate_neighboring_clusters <- function(clusters, neighborhood_matrix) {
  # Get unique cluster labels
  unique_clusters <- unique(clusters)

  # Initialize an empty list to store neighboring clusters for each cluster
  neighboring_clusters <- list()

  # For each cluster, find neighboring clusters based on the neighborhood matrix
  for (cluster in unique_clusters) {
    cluster_obs <- which(clusters == cluster)
    temp_neighbors <- unique(
      clusters[colSums(neighborhood_matrix[cluster_obs, , drop = FALSE]) > 0]
    )

    # Storing the only the neighbors exclude its own cluster
    neighboring_clusters[[as.character(cluster)]] <- temp_neighbors[temp_neighbors != cluste
  }

  return(neighboring_clusters)
}
```

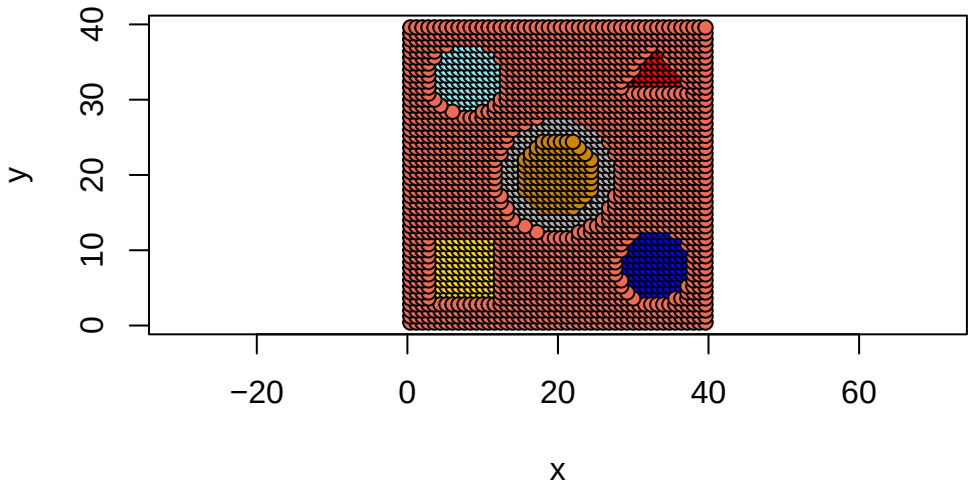
```r
calculate_bi_nb <- function(i, clusters, dist_matrix, neighboring_clusters) {
  # Get the current cluster for observation i
  current_cluster <- clusters[i]

  # Get precomputed neighboring clusters for the current cluster
  neighbors <- neighboring_clusters[[as.character(current_cluster)]]

  # If no neighboring clusters, return Inf for b_i
  if (length(neighbors) == 0) {
    return(list(neighbor = NA, b_i = Inf))
  }

  # Calculate the average distance to observations in neighboring clusters
  b_i_values <- sapply(neighbors, function(cluster) {
    cluster_points <- which(clusters == cluster)
    mean(dist_matrix[i, cluster_points])
  })

  # Get the smallest average distance (min b_i) and corresponding neighboring cluster
  min_b_i <- min(b_i_values)
  neighbor <- neighbors[which.min(b_i_values)]

  return(list(neighbor = neighbor, b_i = min_b_i))
}

calculate_custom_silhouette <- function(clusters, dist_matrix, neighborhood_matrix) {
  # Convert dist object to matrix
  dist_matrix <- as.matrix(dist_matrix)

  # Precompute neighboring clusters for each cluster
  neighboring_clusters <- calculate_neighboring_clusters(clusters, neighborhood_matrix)

  # Sequential computation of silhouette widths and neighbors
  results <- lapply(seq_len(length(clusters)), function(i) {
    a_i <- calculate_ai(i, clusters, dist_matrix)
    b_i_nb <- calculate_bi_nb(i, clusters, dist_matrix, neighboring_clusters)
    sil_width <- (b_i_nb$b_i - a_i) / max(a_i, b_i_nb$b_i)

    # Return silhouette width and neighbor
    list(sil_width = sil_width, neighbor = b_i_nb$neighbor)
  })
```

```
  # Extract silhouette widths and neighbors from results
  sil_widths <- sapply(results, function(res) res$sil_width)
  neighbors <- sapply(results, function(res) res$neighbor)

  # Combine results into a matrix for output
  wds <- cbind(
    cluster = clusters,
    neighbor = neighbors,
    sil_width = sil_widths
  )

  attr(wds, "Ordered") <- FALSE
  attr(wds, "call") <- match.call()
  class(wds) <- "silhouette"

  return(wds)
}
```

Performing constrained hierarchical clusteing

```
# Distance based on features
feature_dist <- dist(st_drop_geometry(data_regions[,2:(p+1)]))

# Create new a new neighborhood structure
knn <- 7
sf_df_coords <- st_coordinates(data_regions)
nb <- knn2nb(knearneigh(sf_df_coords, k = knn)) %>%
  #make.sym.nb() %>%
  nb2listw(style = "W")

post_hoc_w <- listw2mat(nb)

# Creating the data frame needed for the constr.hclust function
neighbors <- listw2sn(nb)[,1:2]

# Applying constrained h.clustering
constr.clust <- constr.hclust(
  d = feature_dist, method = "ward.D2", links = neighbors, coords = sf_df_coords
)
```

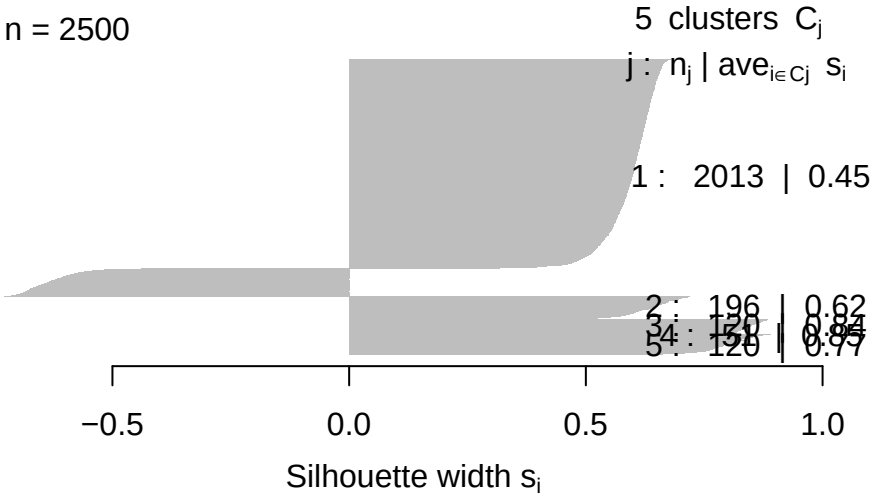
```r
plot(constr.clust, k = 7)
```



Obtaining the silhouette score

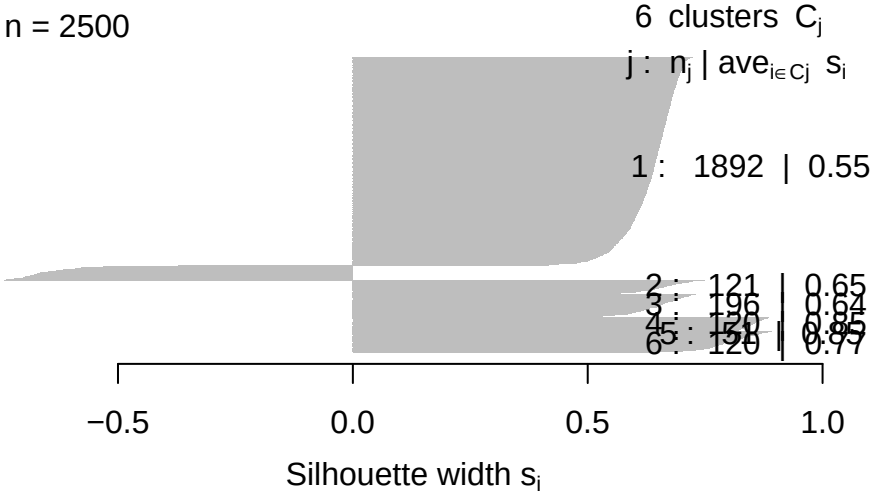
```r
num_clusters <- 5:7
for (i in (num_clusters)){
  clusters <- as.factor(cutree(constr.clust, k = i))
  sil_cl1 <- calculate_custom_silhouette(clusters, feature_dist, listw2mat(nb))
  plot(sil_cl1, main =  paste("Custom Silhouette Plot for", i, "Clusters"), border = NA)
}
```

Custom Silhouette Plot for 5 Clusters

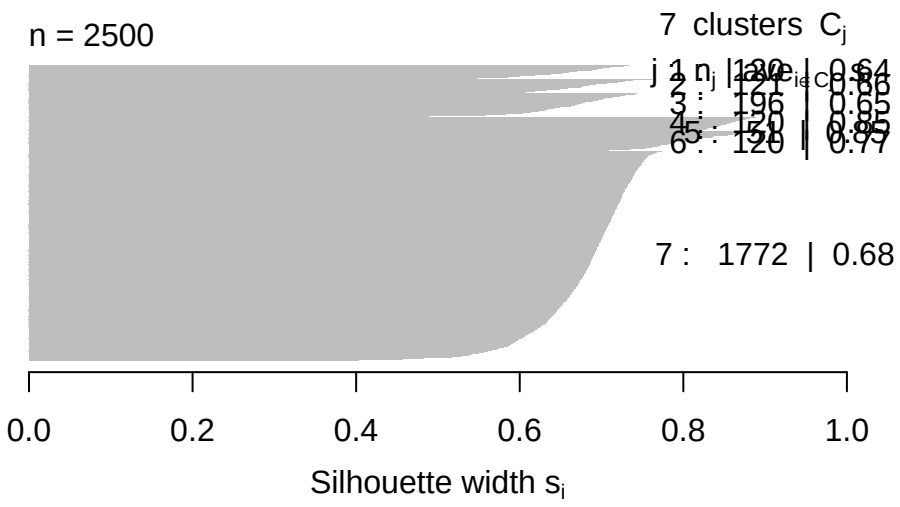


Average silhouette width : 0.5

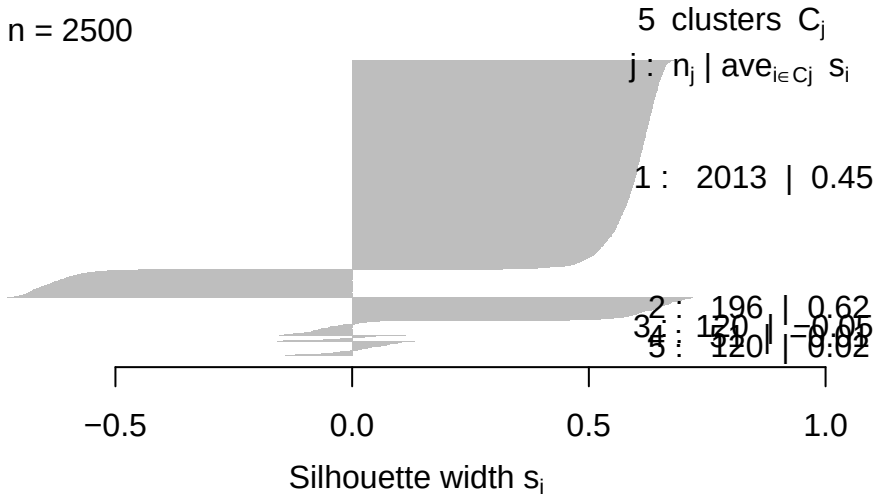
Custom Silhouette Plot for 6 Clusters



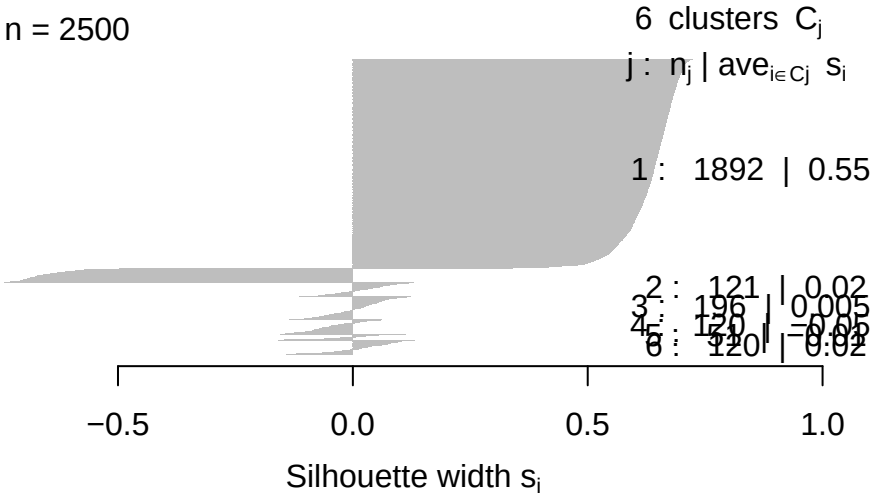
Average silhouette width : 0.59




```
num_clusters <- 5:7
for (i in (num_clusters)){
  clusters <- cutree(constr.clust, k = i)
  sil_cl2 <- silhouette(clusters, feature_dist, listw2mat(nb))
  plot(sil_cl2, main =  paste("Custom Silhouette Plot for", i, "Clusters"), border = NA)
}
```

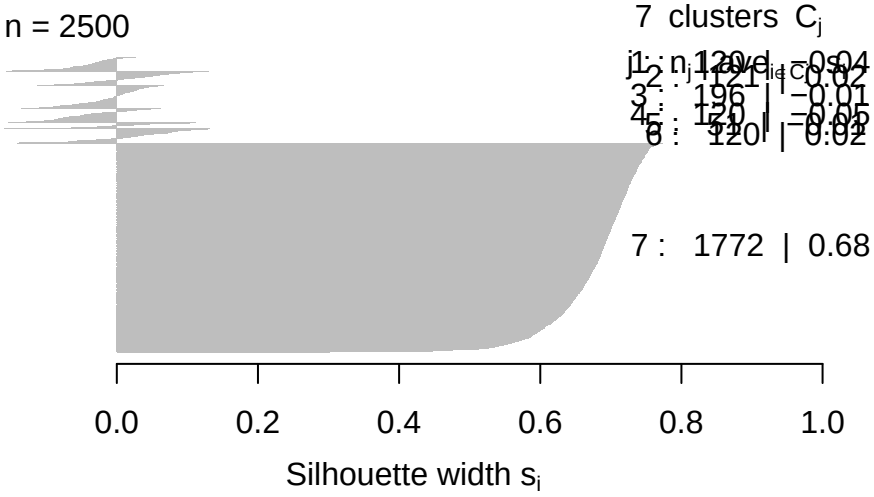


Custom Silhouette Plot for 6 Clusters



Average silhouette width : 0.42

Custom Silhouette Plot for 7 Clusters



Average silhouette width : 0.48

Plotting the modified silhouette score

```
clusters <- cutree(constr.clust, k = 7)
sil_cl1 <- calculate_custom_silhouette(clusters, feature_dist, listw2mat(nb))
colors <-  c("#E69F00", "#56B4E9", "#009E73", "#CC79A7","#0072B2", "#D55E00", "#F0E442")
```

```
png("custom_sil.png", width = 7, height = 5, units = "in", res = 300)
plot(sil_cl1,col=colors[clusters], main="", border = NA, do.clus.stat = FALSE)

dev.off()
```

```
pdf
  2
```

Plotting the standard silhouette score

```
clusters <- cutree(constr.clust, k = 7)
sil_cl2 <- silhouette(clusters, feature_dist, listw2mat(nb))
colors <-  c("#E69F00", "#56B4E9", "#009E73", "#CC79A7","#0072B2", "#D55E00", "#F0E442")

png("std_sil.png", width = 7, height = 5, units = "in", res = 300)
plot(sil_cl2,col=colors[clusters], main="", border = NA, do.clus.stat = FALSE)

dev.off()
```

```
pdf
  2
```

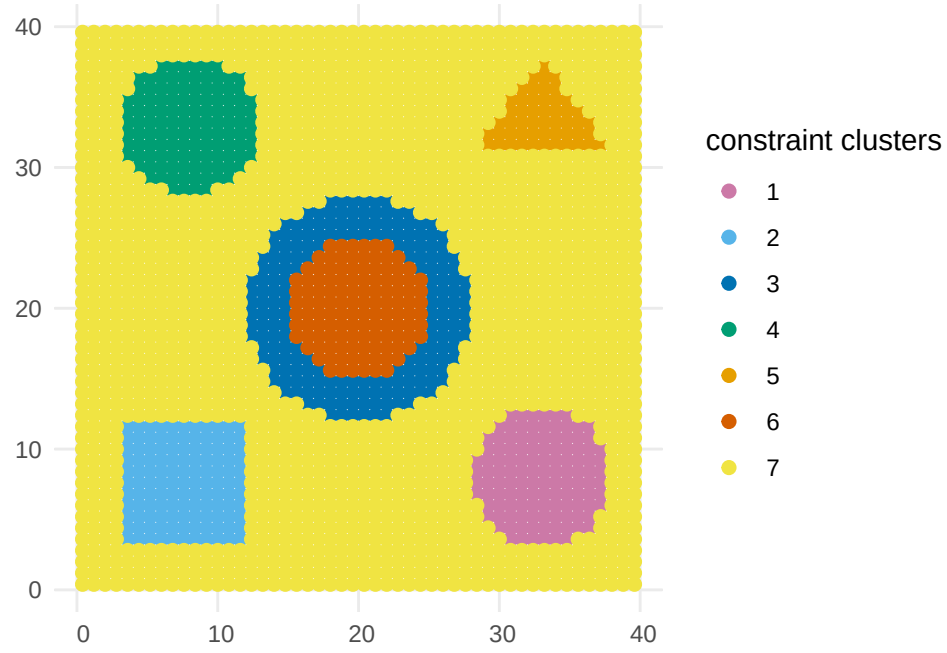
```
data_regions$`constraint clusters` <- as.factor(cutree(constr.clust, k = 7))
```

Plotting the resultant clusters

```
cb_palette <- c("#CC79A7","#56B4E9", "#0072B2", "#009E73", "#E69F00", "#D55E00","#F0E442")

g3 <- ggplot() +  # Plot captured shapes
  geom_sf(data = data_regions, aes(geometry = coords,, color = `constraint clusters`), size =
  theme_minimal() +
  scale_color_manual(values = cb_palette) +
  labs(fill = "Regions") +
  theme(legend.position = "right")
```

```
g3
```

```
Ignoring unknown labels:
* fill : "Regions"
```




```
#ggsave("Constrained Clusters - Sim.png", plot = g3, width = 7, height = 5, units = "in")
```